SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA0713 **COURSE NAME: COMPUTER NETWORKS**

COURSE OUTCOME COVERED

CO2: Configure IP addressing schemes, subnetting, and basic routing techniques using simulation tool, for efficient network design and topology. *(BL3)*

Assessment Tool Weightage: 40%

Annexure A

ANALYTICAL PROBLEM SOLVING

Code of Conduct:

I am **B Tharun Kumar/192424356** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: B Tharun Kumar

Annexure A

ANALYTICAL PROBLEM SOLVING

1. Suppose we want to transmit the message 11100011 and protect it from errors using the CRC polynomial X3+1.

i.) Use polynomial long division to determine the message that should be transmitted.

ii) Suppose the leftmost bit of the message is inverted due to noise on the transmission link. What is the result of the receiver's CRC calculation? How does the receiver know that an error has occurred?

Answer 1:

(a) Problem Understanding

We are given a message M = 11100011 (8 bits) that must be protected against transmission errors using the Cyclic Redundancy Check (CRC) technique. The generator polynomial supplied is G(x) = X3 + 1, which as a bit string is 1001 (degree r = 3, since the highest power of x present is 3). The task has two parts: (i) find the actual bit sequence that must be transmitted (message + CRC), and (ii) analyse what happens at the receiver if the left-most bit of this transmitted sequence gets flipped in transit.

(b) Method Selection

CRC works on modulo-2 (XOR) polynomial division, not ordinary arithmetic division, because bits behave as coefficients over GF(2). The standard CRC procedure is: append r zero bits (r = degree of generator) to the message, divide the result by the generator using XOR, and attach the remainder (called the Frame Check Sequence, FCS) to the original message. This method is selected because it is the universally used, industry-standard technique (used in Ethernet, HDLC, etc.) and it exactly matches what the question asks for -- 'polynomial long division'.

(c) Solution Process -- Step by step

Step 1: Append r = 3 zero bits to the message.

M appended = 11100011 000 = 11100011000 (11 bits)

Step 2: Divide 11100011000 by the generator 1001 using XOR (modulo-2) division. The division proceeds bit by bit; at each step, if the leading bit of the current 4-bit window is 1, XOR the window with 1001, otherwise XOR with 0000, then shift in the next bit.

Step	4-bit window	Leading bit	XOR result
1	1110	1	0111
2	1110	1	0111
3	1110	1	0111
4	1111	1	0110
5	1101	1	0100
6	1000	1	0001
7	0010	0	0010
8	0100	0	0100

Quotient obtained = 11111100 (not required, shown only for completeness). The last 3 bits of the final result give the remainder:
Remainder (CRC) = 100

Transmitted message = Message + CRC = 11100011 + 100 = 11100011100

(d) Accuracy Check / Verification

A correct CRC must divide the full transmitted codeword exactly, leaving a remainder of 000. Dividing 11100011100 by 1001 using the same XOR procedure indeed gives remainder 000, confirming that the encoding in Step 2 is correct.

Part (ii): Error introduced at the receiver

The left-most bit of the transmitted codeword 11100011100 is inverted (1 to 0) by noise, so the receiver actually receives: 01100011100.
The receiver performs exactly the same division: it divides the received bit stream 01100011100 by the generator 1001. Carrying out the XOR division gives a remainder of 010.

(e) Interpretation of Result

Since the remainder computed by the receiver (010) is NOT equal to 000, the receiver immediately knows that the received codeword is not exactly divisible by the generator polynomial. A non-zero remainder is the signal used by CRC to flag corruption -- the receiver does not need to know which bit is wrong, only that at least one bit changed, so it discards the

frame and (typically) requests retransmission. This confirms that the single-bit inversion in this scenario is successfully detected by the CRC-3 scheme.

2. a) A network with bandwidth of 10 Mbps can pass only an average of 12,000 frames per minute with each frame carrying an average of 10,000 bits. What is the throughput of this network?

b) A network has a bandwidth of 3000 Hz (300 to 3300 Hz) assigned for data communications. The signal- to-noise ratio is 3162. Find the channel capacity

Answer 2:

Part (a): Throughput

Problem Understanding:

We are told the physical bandwidth of the link (10 Mbps) and the actual rate at which useful frames are delivered (12,000 frames/minute, 10,000 bits/frame). Throughput is the metric that measures how much data is actually and successfully delivered per unit time -- it is generally lower than the raw bandwidth because of overheads, delays and protocol inefficiencies.

Method Selection:

Throughput = (number of frames delivered per second) x (bits carried per frame). This formula is chosen because it directly converts the given per-minute frame rate into an effective bit rate, which is what 'throughput' means.

Solution Process:

Frames per second = 12,000 frames / 60 seconds = 200 frames/sec

Throughput = 200 frames/sec x 10,000 bits/frame = 2,000,000 bits/sec = 2 Mbps

Interpretation:

Even though the channel is physically capable of 10 Mbps, the network only manages to actually push through 2 Mbps of useful data. Throughput will always be less than or equal to bandwidth; the gap of 8 Mbps here indicates significant overhead, congestion, or an application/processing bottleneck limiting the effective data delivery rate.

Part (b): Channel Capacity

Problem Understanding:

We must find the theoretical maximum error-free data rate (channel capacity) of a noisy channel with bandwidth B = 3000 Hz (300 Hz to 3300 Hz, a voice-grade line) and signal-to-noise ratio SNR = 3162.

Method Selection:

Because the channel is noisy and we are given an SNR value (not just bandwidth), Shannon's Channel Capacity Theorem is the correct and only applicable formula: C = B log2(1 + SNR). (Nyquist's formula would apply only for a noiseless channel, which is not the case here.)

Solution Process:

C = B log2(1 + SNR)

C = 3000 x log2(1 + 3162)

C = 3000 x log2(3163)

log2(3163) = 11.627 (approximately)

C = 3000 x 11.627 = 34,881 bps (approximately)

C is approximately equal to 34.88 kbps

Accuracy Check:

Note that SNR = 3162 corresponds almost exactly to an SNR in decibels of 10 log10(3162) = 35 dB, a realistic value for a telephone line, which cross-checks that the given numbers are consistent with a standard textbook voice-channel example.

Interpretation:

This value, 34.88 kbps, is the absolute upper limit on error-free data transmission achievable on this 3000 Hz noisy channel -- no modulation scheme, however clever, can reliably exceed this rate on this channel. This is why classical analog telephone modems topped out at speeds close to this figure (e.g., 33.6 kbps / 56 kbps modems).

3. A multispecialty hospital uses wireless patient-monitoring devices to continuously transmit patients' heart rate and blood pressure readings to a central monitoring station. Since the data is transmitted over a wireless channel, electromagnetic interference occasionally changes one bit during transmission. To ensure that doctors receive accurate patient information without waiting for retransmission, the hospital implements the **Hamming (12,8)** error-correcting code.

Answer 3:

(a) Problem Understanding

A hospital sends critical 8-bit patient vitals over a wireless channel where occasional single-bit errors occur. Because waiting for retransmission is unacceptable for real-time monitoring, the system needs a code that can correct errors on the fly, not just detect them. The question specifies the Hamming (12, 8) code, meaning 8 data bits are carried inside a 12-bit codeword (so r = 4 redundant/parity bits are added).

(b) Method Selection

The Hamming code is chosen (rather than a simple parity bit or CRC) specifically because it is a Single-Error-Correcting code: it does not just flag that an error occurred, it identifies the exact bit position in error so the receiver can flip it back and use the data immediately, with no retransmission. This matches the number of redundant bits required: for k = 8 data bits, the smallest r satisfying 2^r >= k + r + 1 is r = 4 (since 2^4 = 16 >= 13), giving n = k + r = 12 total bits -- exactly the Hamming(12,8) code named in the question.

(c) Solution Process

Bit placement rule: parity bits are placed at all power-of-2 positions (1, 2, 4, 8) and data bits fill the remaining positions (3, 5, 6, 7, 9, 10, 11, 12), counting from the left as position 1.

Position	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**
Bit type	P1	P2	d1	P4	d2	d3	d4	P8	d5	d6	d7	d8

Each parity bit checks (covers) a fixed set of positions, determined by binary weighting -- a position is covered by Pn if the binary representation of that position has the corresponding bit set:

P1 (checks bit 1 of the binary position number) covers positions: 1, 3, 5, 7, 9, 11

P2 (checks bit 2) covers positions: 2, 3, 6, 7, 10, 11

P4 (checks bit 3) covers positions: 4, 5, 6, 7, 12

P8 (checks bit 4) covers positions: 8, 9, 10, 11, 12

Each parity bit is set so that the total number of 1s in its covered group (including itself) is even -- this is called even parity.

Worked numerical illustration

(using the 8-bit data 11100011, the same message given in Question 1, placed left-to-right into positions 3, 5, 6, 7, 9, 10, 11, 12):

d1=1 (pos 3), d2=1 (pos 5), d3=1 (pos 6), d4=0 (pos 7), d5=0 (pos 9), d6=0 (pos 10), d7=1 (pos 11), d8=1 (pos 12).

P1 = d1 XOR d2 XOR d4 XOR d7 = 1 XOR 1 XOR 0 XOR 1 = 1

P2 = d1 XOR d3 XOR d4 XOR d7 = 1 XOR 1 XOR 0 XOR 1 = 1

P4 = d2 XOR d3 XOR d4 XOR d8 = 1 XOR 1 XOR 0 XOR 1 = 1

P8 = d5 XOR d6 XOR d7 XOR d8 = 0 XOR 0 XOR 1 XOR 1 = 0

Final 12-bit transmitted codeword (positions 1-12): 1 1 1 1 1 1 0 0 0 0 1 1 i.e. 111111000011

(d) Accuracy of Calculation -- Error Correction Demonstration

Suppose interference flips the bit at position 7 during transmission: the receiver actually gets 111111100011.

The receiver independently recomputes the four parity checks over the received bits. Whichever checks fail (parity becomes odd) contribute their position-value to a running total called the syndrome:

Check P1 (positions 1,3,5,7,9,11): fails -> contributes 1

Check P2 (positions 2,3,6,7,10,11): fails -> contributes 2

Check P4 (positions 4,5,6,7,12): fails -> contributes 4

Check P8 (positions 8,9,10,11,12): passes -> contributes 0

Syndrome = 1 + 2 + 4 + 0 = 7

(e) Interpretation of Result

The syndrome value (7) exactly identifies position 7 as the location of the corrupted bit -- this is precisely where the error was introduced, confirming the method works correctly. The receiver simply inverts bit 7 to restore the original, correct codeword 111111000011, recovering the patient's data (11100011) without needing any retransmission. This self-correcting property is exactly why Hamming(12,8) is well-suited to a real-time, latency-sensitive application such as continuous wireless patient monitoring: a single-bit glitch caused by electromagnetic interference is fixed instantly and transparently, with no delay in delivering vital readings to doctors.

4. a) A company uses the IP address **172.16.10.0/24** and has implemented subnetting using a **/27 subnet mask** for its internal departments. Using the given subnet mask, apply subnetting techniques to calculate the total number of subnets created and the number of usable hosts per subnet. Determine the subnet to which the IP address **172.16.10.78** belongs. Identify the network address and broadcast address for that subnet. Also, determine whether the IP address **172.16.10.95** falls within the same subnet range based on the given configuration.

b) A network uses the Internet Protocol version 6 address **2001:0db8:0000:0000:0000:ff00:0042:8329**. Apply IPv6 addressing rules to compress the given address into its shortest form and then expand the compressed address back to its full notation. Using a prefix length of **/64**, determine the network prefix of the given address. Also, calculate the total number of possible host addresses available within this network based on the given prefix length.

Answer 4:

Part (a): IPv4 Subnetting

Problem Understanding:

We start with the network 172.16.10.0/24 (a full Class B-style /24 block, 256 addresses) and must re-divide it into smaller subnets using a /27 mask for departmental separation. We then need to locate two specific host addresses (172.16.10.78 and 172.16.10.95) within this new subnet scheme.

Method Selection:

Standard subnetting formulas are used: borrowed bits = new prefix - old prefix; number of subnets = 2^(borrowed bits); host bits = 32 - new prefix; usable hosts per subnet = 2^(host bits) - 2 (two addresses, network and broadcast, are reserved in every subnet). The block size (subnet increment) is found as 2^(host bits), which lets us list subnet boundaries quickly instead of converting every address to binary.

Solution Process:

Original prefix = /24; new prefix = /27, so bits borrowed = 27 - 24 = 3

Number of subnets = 2^3 = 8

Remaining host bits = 32 - 27 = 5

Usable hosts per subnet = 2^5 - 2 = 32 - 2 = 30

Block size (increment) = 2^5 = 32, so each subnet spans 32 consecutive addresses in the last octet.

The 8 subnets are listed below, together with their network address, broadcast address and usable host range:

Subnet #	**Network Address**	**Usable Host Range**	**Broadcast Address**
1	172.16.10.0	172.16.10.1 - .30	172.16.10.31
2	172.16.10.32	172.16.10.33 - .62	172.16.10.63

3	172.16.10.64	172.16.10.65 - .94	172.16.10.95
4	172.16.10.96	172.16.10.97 - .126	172.16.10.127
5	172.16.10.128	172.16.10.129 - .158	172.16.10.159
6	172.16.10.160	172.16.10.161 - .190	172.16.10.191
7	172.16.10.192	172.16.10.193 - .222	172.16.10.223
8	172.16.10.224	172.16.10.225 - .254	172.16.10.255

Locating 172.16.10.78:

Since 64 <= 78 <= 95, this address lies in Subnet #3 (172.16.10.64/27).

Network Address = 172.16.10.64

Broadcast Address = 172.16.10.95

Usable Host Range = 172.16.10.65 to 172.16.10.94

Checking 172.16.10.95:

This address equals the broadcast address of the same subnet (172.16.10.64/27, range 64-95). So yes, 172.16.10.95 does fall within the same subnet range as 172.16.10.78 -- however, being the broadcast address, it is reserved for subnet-wide broadcasts and cannot itself be assigned to a host device.

Interpretation:

The original single /24 network is now cleanly split into 8 isolated departmental subnets of 30 usable hosts each, which supports better traffic isolation, security and address management than one large flat network.

Part (b): IPv6 Addressing

Problem Understanding:

We are given the IPv6 address 2001:0db8:0000:0000:0000:ff00:0042:8329 and must (i) compress it to its shortest legal form, (ii) expand it back to full notation, (iii) find the network prefix for a /64 mask, and (iv) count the total host addresses available in that /64 network.

Method Selection:

IPv6 compression rules are applied: leading zeros within each 16-bit block may be dropped, and the single longest run of consecutive all-zero blocks may be replaced once with '::'. The network prefix is obtained by keeping the first (prefix-length / 16) blocks and zeroing the rest;

the number of host addresses in a subnet is 2 raised to the number of remaining host bits (128 - prefix length).

Solution Process:

Original: 2001:0db8:0000:0000:0000:ff00:0042:8329

Step 1 (drop leading zeros in each block): 2001:db8:0:0:0:ff00:42:8329

Step 2 (compress the longest run of all-zero blocks, here three consecutive zero blocks, with '::'):

Compressed form = 2001:db8::ff00:42:8329

Expanding the compressed address back: each block is restored to 4 hex digits and the '::' is re-expanded to the correct number of all-zero blocks (here 3 blocks) to make a total of 8 blocks:

Expanded form = 2001:0db8:0000:0000:0000:ff00:0042:8329

Network Prefix for /64: the first 64 bits (the first four 16-bit blocks) define the network, and the remaining 64 bits are the host/interface portion:

Network Prefix = 2001:0db8:0000:0000::/64 (compressed: 2001:db8::/64)

Total possible host addresses = 2^(128 - 64) = 2^64

2^64 = 18,446,744,073,709,551,616, i.e. approximately 1.8 x 10^19 addresses.

Interpretation:

A single /64 IPv6 subnet can address roughly 18.4 quintillion hosts -- vastly more than could ever be practically used, which is intentional in IPv6 design: it simplifies auto-configuration (e.g., SLAAC, which relies on a full 64-bit interface identifier) rather than conserving address space, unlike the scarce, tightly-managed IPv4 space seen in part (a).

5. A university is conducting an online semester examination for **8,000 students** simultaneously. At exactly **10:00 AM**, the examination server begins transmitting encrypted question papers to all students. The university server is connected through a **10 Gbps high-speed network**, while students access the examination portal using different internet connections such as **fiber broadband (100 Mbps), home Wi-Fi (20 Mbps), and mobile networks (5–10 Mbps)**. During the first few minutes of the examination, many students with slower internet connections report incomplete downloads, repeated retransmissions, delayed acknowledgments, and temporary freezing of the examination portal. Network administrators observe that the server continues transmitting data at a high rate even though several student

devices are unable to receive and process packets at the same speed. This results in receiver buffer overflow, unnecessary retransmissions, increased network traffic, and delays in delivering the question papers. **Analyze** the above scenario and identify the primary networking problem. Explain how the mismatch between the sender's transmission speed and the receiver's processing capability affects communication.

Answer 5:

(a) Problem Understanding

8,000 students simultaneously connect to an examination server over widely different last-mile links: fiber (100 Mbps), home Wi-Fi (20 Mbps) and mobile (5-10 Mbps), while the server itself sits on a 10 Gbps backbone. Symptoms reported are incomplete downloads, repeated retransmissions, delayed ACKs and portal freezing -- and the administrators specifically observe that the server keeps sending at high speed even though many student devices cannot keep up. This is the key clue that must be analysed.

(b) Method Selection -- Identifying the Primary Problem

The symptom pattern described (fast sender, slow receiver, buffer overflow at the receiver, retransmissions caused by data loss due to overwhelmed receivers) is the textbook signature of a Flow Control problem, i.e. a speed/capacity mismatch between sender and receiver, and NOT primarily a routing, addressing or physical-layer problem. Because the mismatch is happening simultaneously across 8,000 independent receivers sharing the same network core, a secondary, related issue -- network Congestion -- is also present and must be discussed alongside flow control.

(c) Solution Process -- Cause and Effect Chain

The breakdown can be explained as a chain of cause and effect:

1. The server transmits question papers at a rate suited to its own 10 Gbps interface, without initially throttling to each individual student's much slower access link.
2. Each receiving device has a finite buffer to hold incoming segments before the application (browser/exam portal) can read and process them.
3. On slow links (home Wi-Fi 20 Mbps, mobile 5-10 Mbps), incoming data arrives faster than the device/application can drain its buffer, so the buffer fills up and overflows -- excess packets are silently dropped.

4. Dropped packets are never acknowledged by the receiver, so after a timeout the sender (following normal reliable-transport behaviour) retransmits them, adding even more traffic onto already-strained links.
5. Because thousands of students are experiencing this at once, the aggregate retransmission traffic adds to congestion in the shared network core, delaying ACKs further for everyone -- including students on otherwise adequate connections.
6. The visible result is exactly what was reported: incomplete downloads, repeated retransmissions, delayed acknowledgments and a frozen exam portal.

(d) Interpretation of Results

The mismatch between the sender's transmission speed and the receiver's processing/reception capability is precisely what flow control is designed to prevent. In protocols such as TCP, this is solved using the Sliding Window mechanism: each receiver advertises a receive window (rwnd) in every ACK, telling the sender exactly how many unacknowledged bytes it may still send. The sender is contractually bound to never exceed this window, so a slow mobile-network student automatically causes the server to slow its transmission specifically to that student, instead of overwhelming their buffer.

In parallel, because the trouble here also stems from network-wide congestion (many senders/receivers competing for the same core links, not just one sender-receiver pair), TCP Congestion Control mechanisms -- slow start, congestion avoidance, and multiplicative decrease on packet loss (AIMD) -- are equally necessary. These allow the server to probe available capacity gradually for each connection and back off quickly when losses are detected, rather than blindly pushing data at full speed to every one of the 8,000 students at once.

(e) Recommendation

For an exam platform at this scale, the university should ensure: (i) proper TCP flow-control/congestion-control tuning on the server, (ii) staggering or batching of exam-paper delivery (e.g., releasing papers in smaller chunks or staggered start times) to avoid a synchronised burst of 8,000 simultaneous high-volume transfers, and (iii) adequate server-side and CDN-level buffering/back-off logic so that slower student connections are served at a sustainable rate instead of causing cascading retransmissions across the whole network.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation